

软件过程与管理课程作业（Assgt_01）

项目软件过程定义、来源与剪裁说明

作业编号	Assgt_01	作业性质	课后作业
学生姓名	王涵	学号	2023141461082
提交文件名	2023141461082-王涵-第一次课后作业.pdf	课程	软件过程与管理

一、作业理解与阅读概述

本次作业要求在大致阅读 ISO/IEC/IEEE 12207、ISO/IEC 15504、ISO/IEC 33001 等资料的基础上，结合自己做过的项目定义一个软件过程，并说明该过程定义的来源与剪裁思路。我的理解是：作业重点不只是罗列一个开发流程，而是要说明“为什么这样定义”“哪些过程来自标准或课程内容”“在具体项目规模和约束下如何取舍”。

ISO/IEC/IEEE 12207 提供了软件生命周期过程的总体框架，其过程可从协议过程、组织项目使能过程、技术管理过程和技术过程等角度组织；ISO/IEC 15504 与 ISO/IEC 33001 更强调过程评估、过程能力、过程属性、证据与改进。结合课程中“过程具有层次性、可定义、可剪裁”的思想，本文以一个小型校园二手交易系统为例，定义适合小团队课程项目的软件过程。

二、项目背景

项目名称：校园二手交易系统。

项目类型：Web/移动端小型应用，面向校内学生提供二手书籍、数码配件、生活用品的信息发布、搜索浏览、私信联系、订单状态管理和举报反馈等功能。

项目规模与环境：4 人小组开发，周期约 6 周。前端使用 Vue 或微信小程序风格界面，后端使用 Spring Boot 或 Node.js，数据库使用 MySQL。项目不是商业级产品，但需要完成从需求到交付的完整闭环，具备一定的用户角色、权限、数据管理和质量控制要求。

主要干系人包括：学生用户、系统管理员、开发小组、课程教师/助教。项目目标是交付一个可演示、可部署、需求可追踪、质量可说明的软件原型。

三、软件过程总体定义

本项目采用“轻量级迭代增量过程”。整个项目分为启动与计划、需求分析、架构与设计、迭代实现、验证确认、交付总结六个阶段。每个阶段都有明确输入、活动、输出和评审点。

过程总体原则如下：第一，以需求和可交付物为中心，确保每次迭代都形成可运行版本；第二，以文档适度化为原则，保留必要证据，不做过度形式化；第三，以风险优先和质量内建为原则，将关键风险、接口、数据库和核心业务流程提前验证；第四，以可追踪和可改进为原则，需求、任务、代码提交、缺陷和测试结果之间保持基本关联。

四、过程分解与活动定义

为了体现过程定义的层次性，本文将项目过程分为阶段、子过程、主要活动、输入/输出和质量控制点五个层次。

阶段/子过程	主要活动	输入	输出/工作产品	控制点
1. 启动与项目计划	明确项目目标、范围、角色分工、里程碑、沟通方式和风险清单；建立 Git 仓库和任务看板。	课程要求、项目设想、团队资源	项目计划、角色分工表、WBS、里程碑计划、初始风险清单	教师/组内评审计划是否可执行
2. 需求分析	访谈或模拟目标用户，整理功能需求与非功能需求；编写用例、用户故事和验收标准。	用户场景、课程目标、竞品功能	需求规格说明、用例图/用户故事、需求优先级列表	需求是否清晰、可测试、可追踪
3. 架构与设计	确定系统架构、模块划分、数据库表、接口契约和页面原型；识别关键技术风险。	需求规格、技术约束	概要设计、数据库设计、接口文档、页面原型	接口和数据库是否支持核心业务流程
4. 迭代实现	按优先级开发登录注册、商品发布、搜索浏览、订单/联系、后台管理等功能；进行代码评审和持续集成。	设计文档、任务看板	源代码、提交记录、构建包、迭代版本	代码能否构建运行，核心功能是否完成
5. 验证与确认	执行单元测试、接口测试、系统测试和用户验收测试；记录缺陷并回归。	需求、测试用例、迭代版本	测试用例、缺陷清单、测试报告、验收记录	缺陷关闭率、关键用例通过率
6. 交付与总结	整理部署说明、用户说明、项目总结和改进建议；完成最终演示。	最终版本、测试报告、项目记录	发布包、部署说明、用户手册、项目总结报告	交付物完整，可复现部署

五、关键支持过程定义

1. 配置管理过程：以 Git 仓库作为配置项管理中心。main 分支保存稳定版本，dev 分支用于集成开发，功能分支用于个人任务。需求文档、设计文档、接口文档、数据库脚本和测试用例均纳入版本管理。每次合并前要求提交信息清晰，能够对应任务编号。

2. 质量保证过程：质量保证不单独设立重型 QA 岗位，而是由全体成员共同承担。主要做法包括需求评审、设计评审、代码走查、核心接口测试、关键用例验收、缺陷回归和演示前检查。质量目标包括核心业务流程可运行、重要数据不丢失、主要页面无明显错误、接口返回一致。

3. 风险管理过程：项目启动时建立风险清单，每周更新。主要风险包括时间不足、需求频繁变化、数据库设计返工、成员进度不一致、部署环境不稳定。对高风险项采用提前验证、预留缓冲、降低范围和备选方案等措施。

4. 度量与监控过程：使用轻量度量指标控制项目状态，包括需求完成率、任务燃尽情况、缺陷数量、关键用例通过率、代码提交频率和版本构建成功率。度量不是为了形式化考核，而是为了发现偏差并及时调整。

5. 变更管理过程：需求变更需记录变更原因、影响范围和处理结论。轻微变更由组内讨论后直接纳入当前迭代；影响数据库、接口或交付时间的变更必须经过组长和相关成员确认，并决定

延期、替换或取消。

六、过程工作产品清单

本项目的软件过程会产生以下主要工作产品：项目计划、WBS 与进度表、需求规格说明、用例/用户故事、数据库设计文档、接口文档、页面原型、源代码、数据库脚本、测试用例、缺陷清单、测试报告、部署说明、用户手册、项目总结报告。这些工作产品共同构成过程执行的客观证据，也便于后续根据 ISO/IEC 15504 或 ISO/IEC 33001 的思想进行过程能力和过程结果评估。

七、软件过程定义的来源

本过程的来源主要有三类。

第一，来源于 ISO/IEC/IEEE 12207 的软件生命周期过程思想。本文将项目过程划分为计划、管理、技术实现、验证确认和交付等活动，并参考其对项目计划、项目评估与控制、风险管理、配置管理、信息管理、度量、质量保证、需求定义、架构设计、实现、集成、验证、确认、运行和维护等过程的组织方式。

第二，来源于 ISO/IEC 15504 和 ISO/IEC 33001 的过程评估思想。本文在定义每个过程时，不只写“做什么”，还说明输入、输出、角色、证据和控制点，使过程能够被观察、被度量、被评价。例如测试报告、缺陷清单、提交记录和评审记录都可以作为过程实施证据。

第三，来源于课程中关于软件过程分类、组成、层次性和剪裁的讲解。项目过程不是固定模板，而是根据项目目标、规模、人员能力、时间限制和风险进行选择。对课程项目而言，过程定义应当既能覆盖生命周期的关键环节，又不能让文档负担超过开发本身。

八、过程剪裁思路

1. 按项目规模剪裁：本项目是 4 人、6 周左右的小型课程项目，因此不采用大型组织中的复杂合同管理、供应商管理、正式审计和多级审批流程，只保留项目计划、任务分配、风险跟踪和交付检查。

2. 按风险剪裁：对风险高的部分加强过程控制，如数据库设计、接口契约、权限控制和核心交易流程；对风险低的部分简化流程，如普通页面样式调整、说明文字修改等只需代码提交和简单自测。

3. 按工作产品剪裁：保留能支撑沟通、开发、测试和验收的文档，取消过度形式化的模板。需求文档不追求长篇幅，而强调需求编号、优先级和验收标准；设计文档重点描述架构、数据库和接口；测试文档重点覆盖核心场景。

4. 按团队能力剪裁：成员经验有限，因此采用清晰、可执行的轻量流程，如任务看板、短周期迭代、每日简短同步、代码评审和演示验收。这样既能降低管理成本，也能保证过程可落地。

5. 按迭代节奏剪裁：项目采用两到三轮迭代。第一轮实现最小可用版本，第二轮补充主要功能，第三轮修复缺陷并完善交付资料。每轮都包含计划、实现、测试、评审和改进。

九、过程执行示例

以“商品发布功能”为例，其过程执行如下：需求阶段明确用户能够上传商品名称、图片、价格、分类、成色和联系方式；设计阶段确定商品表结构、图片存储方式和发布接口；实现阶段由前后端成员分别完成页面和接口；验证阶段根据验收标准检查必填项、价格格式、图片上传、发布成功和异常提示；交付阶段将接口说明、测试结果和部署注意事项写入项目文档。这个例子体现了从需求到设计、实现、验证、交付的端到端过程闭环。

十、过程评估与改进

项目结束后可从过程执行效果进行自评：需求是否完整并被实现，计划是否及时更新，风险是否被识别和处理，代码是否可追溯，测试是否覆盖核心业务，交付物是否可部署。如果发现某些活动只是“写在文档里”而没有产生实际证据，就说明过程定义和执行存在偏差。

改进方向包括：进一步细化需求优先级规则；在 Git 提交信息中统一关联任务编号；增加自动化接口测试；将部署步骤脚本化；在每轮迭代结束时记录经验教训。通过这些改进，可以逐步提升过程的可管理性、可重复性和可评估性。

十一、总结

本文以校园二手交易系统为例，定义了一个适合小型课程项目的轻量级迭代增量软件过程。该过程以 ISO/IEC/IEEE 12207 的生命周期过程为基本框架，以 ISO/IEC 15504 和 ISO/IEC 33001 的过程评估思想作为补充，并结合课程项目的规模和风险进行了剪裁。最终形成的过程既覆盖了需求、设计、实现、验证、交付等核心活动，也包含配置管理、质量保证、风险管理、度量监控和变更管理等支持过程，能够较好地体现软件过程定义的来源、层次性和剪裁思想。

参考资料

ISO/IEC/IEEE 12207:2017, Systems and software engineering - Software life cycle processes.

ISO/IEC 15504, Information technology - Process assessment.

ISO/IEC 33001:2015, Information technology - Process assessment - Concepts and terminology.

《软件过程与管理》课程第一章及课堂讲授内容。